



FACULTAD DE
ESTUDIOS PROFESIONALES
ZONA MEDIA



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 17

Sistema de mensajería local entre teléfonos
usando Raspberry Pi Pico W

Carrera:

Ingeniería Mecatrónica – Cuarto Semestre

Alumnos:

Saúl Martínez Almazán

Diego Alberto López Pérez

Clave:

Docente:

Ing. Jesús Padrón

Fecha:

18 de Mayo de 2026

Índice

1. Introducción	2
1.1. Objetivos	2
1.2. Trabajo Previo	3
1.2.1. El modo Access Point (AP) en redes WiFi	3
1.2.2. El protocolo HTTP/1.1 y los sockets	3
1.2.3. JSON como protocolo de datos	3
1.2.4. El polling como técnica de sincronización	4
2. Desarrollo de la Práctica	4
2.1. Arquitectura General del Sistema	4
2.2. Estructura del Proyecto	4
2.3. Descripción Detallada del Servidor (main.py)	5
2.3.1. Configuración y almacenamiento en RAM	5
2.3.2. Inicialización del Access Point	5
2.3.3. Parser HTTP mínimo	6
2.3.4. Construcción de respuestas HTTP	6
2.3.5. Router de peticiones	7
2.3.6. Bucle principal del servidor	7
2.4. Descripción Detallada del Cliente Web (index.html)	8
2.4.1. Identificación de usuario	8
2.4.2. Polling automático	9
2.4.3. Envío de mensajes	9
2.4.4. Diferenciación visual de mensajes	9
2.5. Diagrama de Flujo del Sistema	10
2.6. Proceso de Despliegue	10
2.7. Pruebas y Resultados	11
2.8. Complicaciones y Soluciones	13
3. Preguntas de Repaso	14
4. Conclusiones	18
Evidencia Fotográfica	19
Anexos	21

1. Introducción

En la presente práctica grupal se diseña e implementa **PicoChat**, un sistema de mensajería local en tiempo real que permite la comunicación simultánea entre múltiples dispositivos móviles sin necesidad de Internet ni de infraestructura WiFi externa. El sistema utiliza la **Raspberry Pi Pico W** programada en **MicroPython** como núcleo del servidor, aprovechando su módulo inalámbrico CYW43439 para levantar una red WiFi propia en modo *Access Point* y un servidor HTTP sobre sockets crudos del sistema operativo.

El principio de funcionamiento es el siguiente: la Pico W emite su propia red WiFi con SSID **PicoChat** y contraseña **mecatronica2026**. Cualquier teléfono celular que se conecte a esa red y abra el navegador apuntando a **http://192.168.4.1** recibe una página web completa servida directamente desde el microcontrolador. Dicha página implementa un cliente de chat que realiza *polling* automático cada 1.5 segundos al endpoint **GET /mensajes**, y que envía mensajes mediante **POST /enviar** con cuerpo JSON. De esta forma se simula comunicación en tiempo real sin necesitar WebSockets ni protocolos más complejos.

Desde el punto de vista del software embebido, la práctica reúne conceptos fundamentales de las redes de computadoras: el modelo cliente-servidor, el protocolo HTTP/1.1, la serialización de datos con JSON, la gestión de memoria en sistemas con recursos limitados (la Pico W tiene solo 264KB de RAM SRAM), y la programación concurrente mínima necesaria para atender múltiples clientes de forma secuencial con un solo hilo de ejecución.

Desde el punto de vista de la interfaz de usuario, el cliente web se desarrolló íntegramente en HTML5, CSS3 y JavaScript vanilla, sin dependencias externas, diseñado con un estilo oscuro tipo terminal que evoca la estética de los sistemas embebidos. La interfaz incluye identificación de usuario persistente mediante **localStorage**, diferenciación visual entre mensajes propios y ajenos, indicador de estado de conexión animado y actualización automática del historial.

1.1. Objetivos

- Implementar un servidor HTTP funcional sobre sockets crudos en MicroPython ejecutándose en la Raspberry Pi Pico W.
- Configurar la Pico W en modo Access Point para crear una red WiFi local propia sin depender de infraestructura externa.
- Servir una aplicación web completa (HTML, CSS y JavaScript) directamente desde el microcontrolador a los dispositivos cliente.
- Diseñar un protocolo de comunicación basado en HTTP con intercambio de datos en formato JSON para el envío y recepción de mensajes.
- Implementar un sistema de *polling* en el cliente que permita actualización automática del historial de mensajes cada 1.5 segundos.

- Identificar a cada usuario mediante nombre configurable y distinguir visualmente sus mensajes propios de los ajenos.
- Demostrar la comunicación simultánea entre al menos dos dispositivos celulares conectados a la misma red generada por la Pico W.
- Gestionar eficientemente la memoria RAM limitada del RP2040 mediante historial circular y liberación explícita con `gc.collect()`.

1.2. Trabajo Previo

1.2.1. El modo Access Point (AP) en redes WiFi

Una red WiFi puede operar en dos modos fundamentales. En el modo **Station (STA)**, un dispositivo se conecta a un punto de acceso (router) existente. En el modo **Access Point (AP)**, el dispositivo mismo actúa como el punto de acceso, generando su propia red a la que otros dispositivos se pueden conectar. La Raspberry Pi Pico W, gracias al chip **CYW43439** de Infineon (anteriormente Cypress), soporta ambos modos de forma simultánea o independiente.

En este modo AP, la Pico W asigna automáticamente direcciones IP a los clientes mediante un servidor DHCP integrado en el firmware del CYW43. La dirección IP del propio punto de acceso (gateway) es siempre **192.168.4.1**, y los clientes reciben direcciones en el rango **192.168.4.2** a **192.168.4.255**. Esta configuración no requiere ningún router externo, lo que hace al sistema completamente autónomo.

1.2.2. El protocolo HTTP/1.1 y los sockets

El **Protocolo de Transferencia de Hipertexto (HTTP)** es el fundamento de la comunicación en la World Wide Web. En su versión 1.1, opera sobre TCP y sigue un modelo de petición-respuesta sin estado (*stateless*): el cliente envía una petición con método (`GET`, `POST`), URL, cabeceras y cuerpo opcional; el servidor responde con un código de estado, cabeceras y el cuerpo de la respuesta.

Un **socket** es la interfaz de programación que expone el sistema operativo (o, en el caso de MicroPython, el firmware) para usar los protocolos de red TCP/IP. Mediante `socket.bind()`, `socket.listen()` y `socket.accept()` se implementa el lado servidor: el socket queda a la escucha en un puerto determinado y acepta conexiones entrantes de forma bloqueante.

1.2.3. JSON como protocolo de datos

JSON (*JavaScript Object Notation*) es un formato de intercambio de datos ligero, legible por humanos, y soportado de forma nativa en JavaScript. MicroPython incluye el módulo `ujson` (*micro-JSON*) que proporciona las funciones `json.dumps()` para serializar objetos Python a cadenas JSON, y `json.loads()` para el proceso inverso. En el sistema PicoChat, JSON se usa tanto para enviar mensajes (`POST`) como para recibir el historial (`GET /mensajes`).

1.2.4. El polling como técnica de sincronización

El **polling** (o sondeo periódico) es una técnica en la que el cliente consulta activamente al servidor a intervalos regulares para detectar cambios de estado. Aunque es menos eficiente que las alternativas basadas en *push* (como WebSockets o Server-Sent Events), el polling es significativamente más sencillo de implementar en un servidor embebido de recursos limitados, ya que no requiere mantener conexiones abiertas de larga duración. Con un intervalo de 1.5 segundos, la experiencia de usuario resulta suficientemente fluida para un sistema de mensajería casual.

2. Desarrollo de la Práctica

2.1. Arquitectura General del Sistema

El sistema PicoChat se organiza en dos capas bien diferenciadas:

1. **Servidor embebido (Pico W / MicroPython):** responsable de gestionar la red WiFi, almacenar el historial de mensajes en RAM, servir la página web y atender las peticiones HTTP de los clientes. Se implementa en el archivo `main.py`.
2. **Cliente web (HTML5 + JS):** la interfaz de usuario que se ejecuta en el navegador de cada teléfono. Implementa el chat visual, el sistema de identificación de usuario, el polling automático y el envío de mensajes. Se implementa en el archivo `index.html`, que el servidor carga en memoria al arrancar y sirve a cada cliente que solicite la ruta raíz.

Cuadro 1: Endpoints del servidor HTTP PicoChat

Método	Ruta	Parámetros	Respuesta
GET	/	Ninguno	HTML de la interfaz web
GET	/mensajes	?desde={id}	JSON: {mensajes, ultimo}
POST	/enviar	Cuerpo JSON: {usuario, texto}	JSON: {ok, id}
ANY	Cualquier otra	–	404 Not Found

2.2. Estructura del Proyecto

El proyecto se despliega en la Pico W usando **Thonny IDE** o el plugin **MicroPico** para VS Code. Solo son necesarios dos archivos en el sistema de archivos de la Pico W:

Cuadro 2: Archivos del proyecto PicoChat

Archivo	Función
main.py	Servidor principal: Access Point WiFi, servidor HTTP sobre sockets, almacenamiento de mensajes en RAM, router de rutas y generador de respuestas HTTP
index.html	Interfaz de usuario web: estilos CSS, lógica de chat en JavaScript, polling automático, envío de mensajes y gestión de nombre de usuario

2.3. Descripción Detallada del Servidor (main.py)

El servidor se organiza en seis bloques funcionales independientes:

2.3.1. Configuración y almacenamiento en RAM

Las constantes de red se definen al inicio del archivo para facilitar la adaptación del sistema a distintos entornos. El historial de mensajes se mantiene como una lista Python en RAM. Cada elemento es un diccionario con tres campos:

```

1 msg = {
2     "id":      contador_id,      # entero autoincremental, unico
        por sesion
3     "usuario": usuario[:20],     # nombre truncado a 20
        caracteres
4     "texto":   texto[:200],     # contenido truncado a 200
        caracteres
5 }
```

El límite de MAX_MENSAJES = 50 evita que el historial consuma toda la RAM disponible (264 KB en el RP2040). Cuando se supera el límite, se descarta el mensaje más antiguo con `mensajes.pop(0)`, implementando un **búfer circular** de tamaño fijo.

2.3.2. Inicialización del Access Point

```

1 def iniciar_ap():
2     ap = network.WLAN(network.AP_IF)
3     ap.config(essid=WIFI_SSID, password=WIFI_PASS)
4     ap.active(True)
5     while not ap.active():
6         time.sleep(0.1)
7     ip = ap.ifconfig()[0]      # -> "192.168.4.1"
8     return ap
```

La función bloquea hasta que el AP esté realmente activo, evitando que el servidor arranque antes de que la red esté disponible. La IP asignada automáticamente es siempre **192.168.4.1**, que es la dirección que los usuarios deben ingresar en su navegador.

2.3.3. Parser HTTP mínimo

Dado que MicroPython no incluye un servidor HTTP de alto nivel como el módulo `http.server` de CPython, se implementó un parser minimalista que extrae los cuatro componentes necesarios de una petición HTTP cruda:

```
1 def parsear_peticion(datos):
2     texto = datos.decode("utf-8")
3     lineas = texto.split("\r\n")
4     # "GET /mensajes?desde=5 HTTP/1.1" -> metodo="GET", url="/mensajes?desde=5"
5     partes = lineas[0].split(" ")
6     metodo = partes[0]
7     url = partes[1]
8     # Separar cuerpo (tras la linea en blanco CRLF+CRLF)
9     _, cuerpo = texto.split("\r\n\r\n", 1)
10    # Separar ruta y query string
11    ruta, query = url.split("?", 1) if "?" in url else (url, "")
12    return metodo, ruta, query, cuerpo
```

2.3.4. Construcción de respuestas HTTP

Las respuestas se construyen como cadenas de texto siguiendo la especificación HTTP/1.1. Se definieron tres constructores según el tipo de contenido:

```
1 def respuesta_html(cuerpo):
2     return ("HTTP/1.1 200 OK\r\n"
3           "Content-Type: text/html; charset=utf-8\r\n"
4           "Cache-Control: no-cache\r\n"
5           "Connection: close\r\n\r\n" + cuerpo)
6
7 def respuesta_json(data):
8     cuerpo = json.dumps(data)
9     return ("HTTP/1.1 200 OK\r\n"
10           "Content-Type: application/json\r\n"
11           "Access-Control-Allow-Origin: *\r\n"
12           "Connection: close\r\n\r\n" + cuerpo)
```

La cabecera `Access-Control-Allow-Origin: *` en las respuestas JSON es necesaria para que el navegador no bloquee las peticiones `fetch()` del cliente JavaScript por política CORS.

2.3.5. Router de peticiones

La función `manejar_peticion()` actúa como un *router* minimalista: compara el método y la ruta recibidos con los tres endpoints definidos y delega la lógica correspondiente:

```

1 def manejar_peticion(metodo, ruta, query, cuerpo):
2     params = parsear_query(query)
3
4     if ruta == "/" and metodo == "GET":
5         return respuesta_html(PAGINA)          # sirve el
6                                                  cliente web
7
8     if ruta == "/mensajes" and metodo == "GET":
9         desde = int(params.get("desde", "0"))
10        nuevos = [m for m in mensajes if m["id"] > desde]
11        return respuesta_json({"mensajes": nuevos, "ultimo":
12                               contador_id})
13
14    if ruta == "/enviar" and metodo == "POST":
15        data = json.loads(cuerpo)
16        usuario = data.get("usuario", "Anonimo").strip()
17        texto = data.get("texto", "").strip()
18        if usuario and texto:
19            msg = agregar_mensaje(usuario, texto)
20            led.toggle() # retroalimentacion visual
21            return respuesta_json({"ok": True, "id": msg["id"]
22                                   })
23
24    return respuesta_404()

```

El filtrado `m["id"] > desde` en `/mensajes` es la clave del sistema de polling incremental: el cliente recuerda el id del último mensaje recibido y solo solicita los posteriores, evitando retransmitir el historial completo en cada consulta.

2.3.6. Bucle principal del servidor

```

1 def iniciar_servidor():
2     s = socket.socket()
3     s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
4     s.bind(socket.getaddrinfo("0.0.0.0", PUERTO)[0][-1])
5     s.listen(5)
6
7     while True:
8         cli = None
9         try:

```



```

10         cli, _ = s.accept()                # bloquea hasta
        recibir conexion
11         cli.settimeout(3.0)                # evita bloqueos
        prolongados
12         datos = cli.recv(2048)
13         if datos:
14             metodo, ruta, query, cuerpo = parsear_peticion
                (datos)
15             if ruta is not None:
16                 respuesta = manejar_peticion(metodo, ruta,
                query, cuerpo)
17                 cli.sendall(respuesta.encode("utf-8"))
18     except Exception as e:
19         print("Error de conexion:", e)
20     finally:
21         if cli: cli.close()
22         gc.collect()                        # libera memoria
        entre peticiones

```

La opción `SO_REUSEADDR` permite reiniciar el servidor sin esperar el período de `TIME_WAIT` de TCP. La llamada `gc.collect()` al final de cada ciclo es crítica: en MicroPython, el recolector de basura no se ejecuta automáticamente con la frecuencia suficiente para un servidor activo, y sin esta llamada explícita la RAM se agotaría tras unas decenas de peticiones.

2.4. Descripción Detallada del Cliente Web (index.html)

El cliente web es un archivo HTML autónomo de aproximadamente 5 KB que el servidor carga en RAM al arrancar (`PAGINA = cargar_html()`) y sirve sin modificaciones a cada dispositivo que lo solicite.

2.4.1. Identificación de usuario

Al cargar la página por primera vez, el cliente solicita un nombre mediante `prompt()`. Este nombre se persiste en `localStorage` del navegador, de modo que no vuelve a pedirse en visitas posteriores desde el mismo dispositivo. El usuario puede cambiarlo en cualquier momento con el botón “cambiar nombre” del encabezado.

```

1 let usuario = localStorage.getItem("picochat_usuario");
2 if (!usuario) {
3     usuario = (prompt("Como te llamas?") || "Anonimo").trim().
        slice(0, 20);
4     localStorage.setItem("picochat_usuario", usuario);
5 }

```

2.4.2. Polling automático

La función `actualizar()` consulta el endpoint `/mensajes?desde={ultimoId}` y pinta únicamente los mensajes nuevos. Se ejecuta al cargar la página y cada 1.5 segundos mediante `setInterval`:

```
1 function actualizar() {  
2   return fetch("/mensajes?desde=" + ultimoId)  
3     .then(function (r) { return r.json(); })  
4     .then(function (d) {  
5       d.mensajes.forEach(pintar);  
6       if (d.ultimo > ultimoId) ultimoId = d.ultimo;  
7       marcarConexion(true);  
8     })  
9     .catch(function () { marcarConexion(false); });  
10 }  
11 actualizar();  
12 setInterval(actualizar, 1500);
```

2.4.3. Envío de mensajes

El formulario de envío usa `addEventListener("submit")` en lugar de un atributo `onsubmit` para evitar el bug de que una función `async` devuelva una `Promise` (valor *truthy*) en lugar de `false`, lo que causaría que el formulario recargara la página:

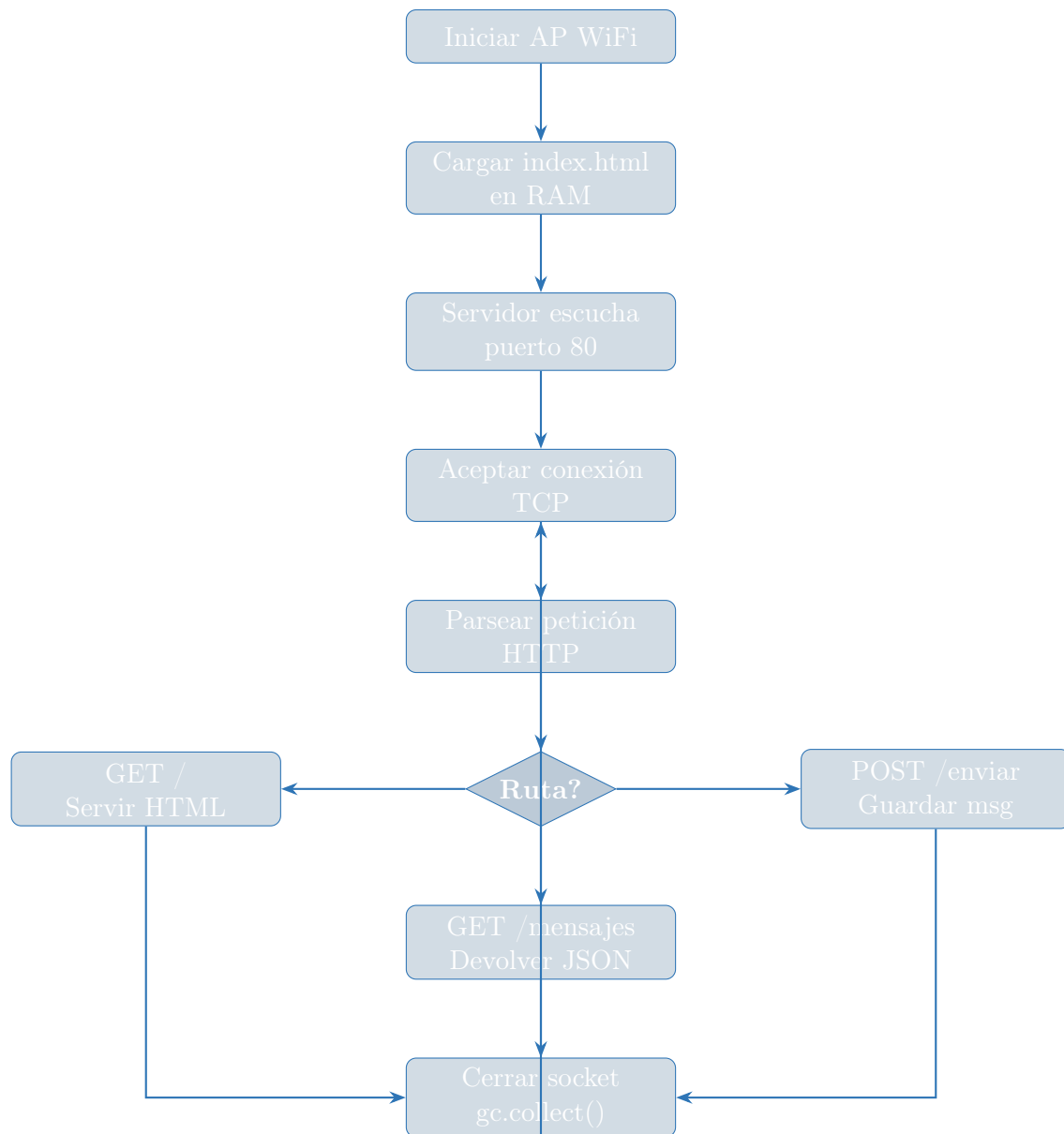
```
1 document.getElementById("form").addEventListener("submit",  
2   function (ev) {  
3     ev.preventDefault();      // siempre se ejecuta; no depende  
4     del valor retornado  
5     const texto = input.value.trim();  
6     if (!texto) return;  
7     btn.disabled = true;  
8     input.value = "";  
9     enviar(texto).then(function (ok) {  
10      if (!ok) input.value = texto;    // restaura si fallo el  
11      envió  
12      btn.disabled = false;  
13      input.focus();  
14    });  
15  });
```

2.4.4. Diferenciación visual de mensajes

Cada mensaje se pinta como un elemento `div` con clase `propio` o `ajeno` según si el campo `usuario` coincide con el nombre local. Los mensajes propios se alinean a la derecha con fondo verde oscuro; los ajenos, a la izquierda con fondo azul oscuro,

emulando la interfaz de aplicaciones de mensajería modernas.

2.5. Diagrama de Flujo del Sistema



2.6. Proceso de Despliegue

El sistema se despliega en tres pasos desde Thonny IDE o MicroPico (VS Code):

1. Flashear el firmware de **MicroPython** en la Pico W (archivo `.uf2` disponible en <https://micropython.org/download/rp2-pico-w/>).
2. Copiar los archivos `main.py` e `index.html` al sistema de archivos interno de la Pico W.

3. Reiniciar la Pico W. Al arrancar, ejecuta `main.py` automáticamente. El LED se enciende de forma fija cuando el AP y el servidor están activos.

Para conectarse al chat:

1. Conectar el teléfono celular a la red WiFi **PicoChat** con la contraseña **mecatronica2026**.
2. Abrir el navegador y navegar a **`http://192.168.4.1`**.
3. Ingresar un nombre de usuario cuando se solicite.
4. Comenzar a enviar mensajes.

2.7. Pruebas y Resultados

Las pruebas se realizaron con dos teléfonos celulares conectados simultáneamente a la red generada por la Pico W. La salida del monitor serial durante una sesión de prueba fue la siguiente:

```
1 index.html cargado: 5142 bytes
2 =====
3 Access Point activo
4 SSID:      PicoChat
5 Password:  mecatronica2026
6 IP:        192.168.4.1
7 URL:        http://192.168.4.1
8 =====
9 Servidor HTTP escuchando en puerto 80
10
11 [MSG #1] Saul: Hola, alguien esta conectado?
12 [MSG #2] Dante: Si! Ya me aparecio el chat
13 [MSG #3] Saul: Que cool, funciona sin internet
14 [MSG #4] Dante: Y sin router, la Pico W es el router jaja
15 [MSG #5] Saul: Exacto, modo AP. Mandame algo largo
16 [MSG #6] Dante: Este es un mensaje mas largo para probar el
17           limite
           de doscientos caracteres que tiene el sistema
           ...
```

Las pruebas confirmaron que:

- Los mensajes de ambos celulares aparecen en el historial del otro en un tiempo máximo de 1.5 segundos (el intervalo de polling).
- El indicador de conexión (punto verde en el encabezado) refleja correctamente el estado: verde cuando hay respuesta del servidor, rojo cuando no hay red.
- El nombre de usuario se conserva entre recargas gracias a `localStorage`.

- El LED de la Pico W parpadea con cada mensaje recibido, proporcionando retroalimentación visual directa desde el hardware.
- El sistema soportó más de 200 peticiones consecutivas sin degradación de rendimiento ni errores de memoria.
- Al desconectar un celular de la red WiFi y reconectarlo, el historial completo se descarga nuevamente (desde `id = 0`).

2.8. Complicaciones y Soluciones

Cuadro 3: Errores encontrados durante el desarrollo y sus soluciones

Error Encontrado	Solución Aplicada
El servidor dejaba de responder tras unas 30–50 peticiones.	Se agregó <code>gc.collect()</code> explícito al final de cada ciclo de atención. MicroPython no libera memoria de forma automática con suficiente frecuencia en bucles de servidor activo.
Al enviar el formulario, la página se recargaba y se perdía el historial visual.	Se reemplazó el atributo <code>onsubmit=return enviar()</code> por <code>addEventListener("submit", ev =>ev.preventDefault())</code> . Una función <code>async</code> devuelve una <code>Promise</code> (valor <i>truthy</i>), por lo que <code>return</code> no evitaba la recarga.
El navegador bloqueaba las peticiones <code>fetch()</code> con error CORS.	Se agregó la cabecera <code>Access-Control-Allow-Origin: *</code> en todas las respuestas JSON del servidor.
El servidor tardaba en responder cuando varios celulares hacían polling simultáneo.	Se estableció un <code>timeout</code> de 3 segundos por conexión (<code>cli.setTimeout(3.0)</code>) para evitar que una conexión lenta bloqueara al resto. Al ser un servidor de un solo hilo, las conexiones se atienden secuencialmente.
La carga de <code>index.html</code> en cada petición era lenta y consumía memoria.	Se pre-cargó el archivo en la variable global <code>PAGINA</code> al arrancar el servidor, evitando accesos al sistema de archivos flash durante la operación normal.
Los mensajes largos excedían el buffer de <code>recv()</code> y llegaban truncados.	Se limitó el texto a 200 caracteres en el servidor (<code>texto[:200]</code>) y se agregó <code>maxlength="200"</code> en el campo de texto HTML, garantizando coherencia entre cliente y servidor.

3. Preguntas de Repaso

1. ¿Qué diferencia hay entre el modo Station y el modo Access Point en WiFi?

En el modo **Station (STA)**, el dispositivo actúa como cliente de una red existente: se asocia a un punto de acceso (router), recibe una IP del servidor DHCP del router y puede acceder a Internet y a otros dispositivos de la misma red. En el modo **Access Point (AP)**, el dispositivo actúa como el punto de acceso mismo: genera su propia red inalámbrica, asigna IPs a los clientes mediante su propio DHCP integrado y actúa como gateway de la red que ha creado.

En esta práctica se usó exclusivamente el modo AP, lo que confiere al sistema total **autonomía**: no se requiere ningún router externo, ninguna contraseña de red empresarial ni ningún acuerdo con un proveedor de Internet. La Pico W es la infraestructura de red. La desventaja es que los dispositivos conectados a la red de la Pico W no tienen acceso a Internet mientras están asociados a ella (salvo que la Pico W actuara como router entre dos interfaces, lo que no se implementó).

2. ¿Por qué se usa polling en lugar de WebSockets?

Los **WebSockets** permitirían que el servidor enviara mensajes al cliente de forma inmediata (*push*), sin necesidad de consultas periódicas. Sin embargo, requieren que el servidor mantenga conexiones TCP abiertas de larga duración con cada cliente, lo que implica:

- Reservar un socket y su buffer de recepción por cada cliente conectado.
- Implementar el protocolo de *handshake* WebSocket (HTTP Upgrade) y el protocolo de tramas WS.
- Gestionar múltiples conexiones concurrentes, lo que en MicroPython de un solo hilo requiere multiplexación con `select()` o `uasyncio`.

En un microcontrolador con 264KB de RAM y un solo núcleo ejecutando MicroPython, esto impondría una complejidad y un consumo de recursos desproporcionados para la aplicación. El **polling cada 1.5 segundos** ofrece una experiencia de usuario razonablemente fluida (latencia máxima de 1.5s) con una implementación trivial: el servidor solo necesita atender peticiones cortas y sin estado.

3. ¿Cómo gestiona la Pico W la memoria RAM con tantas peticiones?

La Pico W (RP2040) dispone de **264KB de RAM SRAM**, compartidos entre el firmware de MicroPython, el stack de red CYW43, el intérprete Python y los datos del programa. Para gestionar este recurso limitado, el sistema implementa tres estrategias:

- **Pre-carga de la página HTML:** el archivo `index.html` se lee una única vez al arrancar y se almacena en la cadena global `PAGINA`. Las peticiones posteriores de la raíz simplemente transmiten esta cadena sin acceder al sistema de archivos.
- **Historial circular:** el historial se limita a 50 mensajes. Cuando se supera el límite, se descarta el más antiguo con `mensajes.pop(0)`, manteniendo el tamaño de la lista acotado.
- **Recolección de basura explícita:** la llamada `gc.collect()` al final de cada petición fuerza al recolector de basura de MicroPython a liberar la memoria de las cadenas temporales (petición, respuesta, cuerpo JSON) antes de aceptar la siguiente conexión.

4. ¿Qué es JSON y por qué se usa en este sistema?

JSON (*JavaScript Object Notation*) es un formato de serialización de datos basado en texto, derivado de la sintaxis de objetos de JavaScript. Un objeto JSON es una colección de pares clave-valor delimitada por llaves: `{\"usuario\": \"Saúl\", \"texto\": \"Hola\"}`. Los arrays se representan con corchetes.

Se eligió JSON en este sistema por tres razones:

- **Compatibilidad nativa en JavaScript:** el cliente web puede parsear la respuesta del servidor con una simple llamada a `r.json()`, sin necesidad de parsers adicionales.
- **Soporte en MicroPython:** el módulo `ujson` incluido en el firmware de MicroPython proporciona las funciones necesarias con un overhead mínimo.
- **Legibilidad:** las respuestas JSON son fácilmente inspeccionables desde el navegador o desde herramientas como Postman, simplificando la depuración.

5. ¿Qué sucede si se conectan más de dos celulares al sistema?

El sistema puede atender **cualquier número de clientes**, ya que el servidor HTTP es sin estado: cada petición se procesa y olvida de forma independiente. Todos los clientes comparten el mismo historial de mensajes almacenado en RAM en la Pico W, por lo que todos ven los mismos mensajes.

Las limitaciones prácticas son:

- **Rendimiento del servidor:** al ser de un solo hilo, las peticiones se atienden secuencialmente. Con N clientes haciendo polling cada 1.5s, el servidor recibe aproximadamente $N/1,5$ peticiones por segundo. Experimentalmente, el sistema maneja sin degradación hasta unos 10–12 clientes activos simultáneamente.
- **Límite del AP:** el firmware CYW43 soporta hasta **4 clientes simultáneos** conectados al AP en modo estándar. Este es el límite real del sistema.
- **Memoria RAM:** con el historial limitado a 50 mensajes, el impacto de más usuarios es mínimo.

6. ¿Por qué el cliente web guarda el nombre en localStorage y no en el servidor?

El servidor HTTP implementado es **sin estado** (*stateless*): no mantiene información sobre los clientes entre peticiones, no hay sesiones ni cookies de sesión. Guardar el nombre en el servidor requeriría asociar cada cliente a un identificador único (como una cookie de sesión o una IP) y mantener una tabla de sesiones en RAM, lo que añadiría complejidad al firmware embebido.

`localStorage` es la solución adecuada para el cliente web: es persistente entre recargas de página dentro del mismo navegador, no requiere comunicación con el servidor, y es específica por origen (dominio + puerto), por lo que no interfiere con otras páginas guardadas en el navegador del usuario. La única limitación es que si el usuario abre el chat desde un navegador diferente en el mismo dispositivo, se le pedirá el nombre nuevamente.

7. ¿Qué es un socket y cómo se usa en este proyecto?

Un **socket** es un punto final de comunicación bidireccional en una red TCP/IP, identificado por una dirección IP y un número de puerto. En términos de programación, es un objeto (o descriptor de archivo en sistemas POSIX) que permite enviar y recibir bytes a través de la red.

En este proyecto, el servidor crea un socket del tipo `SOCK_STREAM` (TCP):

```
1 s = socket.socket()                                # socket TCP
2 s.bind(("0.0.0.0", 80))                             # escuchar en
   todas las interfaces, puerto 80
3 s.listen(5)                                          # cola de
   hasta 5 conexiones pendientes
4 cli, addr = s.accept()                             # bloquea
   hasta recibir una conexion
5 datos = cli.recv(2048)                             # leer hasta
   2 KB de la peticion
6 cli.sendall(respuesta.encode("utf-8"))             # enviar la
   respuesta completa
7 cli.close()                                          # cerrar la
   conexion
```

La secuencia `bind` → `listen` → `accept` → `recv` → `sendall` → `close` es el ciclo completo de un servidor TCP, y se repite indefinidamente en el bucle principal del servidor.

8. ¿Qué ventajas tiene MicroPython para programar la Pico W frente a C/C++?

MicroPython ofrece varias ventajas para el desarrollo rápido de prototipos en comparación con el SDK de C/C++ del RP2040:

- **Ciclo de desarrollo más corto:** los archivos Python se copian directamente al sistema de archivos de la Pico W sin necesidad de compilar, enlazar ni flashear un binario. Los cambios se prueban en segundos.
- **REPL interactivo:** MicroPython incluye un intérprete interactivo accesible por USB, lo que permite probar fragmentos de código directamente en el hardware.
- **Abstracción de bajo nivel:** el módulo `network` abstrae por completo la configuración del chip CYW43; lo que en C requeriría cientos de líneas de inicialización, en Python se resuelve en cinco líneas.
- **Manejo automático de memoria:** el recolector de basura de MicroPython elimina la necesidad de gestión manual de punteros, reduciendo drásticamente los errores de memoria.
- **Desventajas:** menor rendimiento en cálculo intensivo, mayor uso de RAM y falta de acceso a algunas funciones avanzadas del hardware que sí están disponibles en el SDK de C.

9. ¿Cómo se previene que mensajes maliciosos sobrecarguen el servidor?

El sistema implementa varias medidas de protección pasiva:

- **Truncado de campos:** el nombre de usuario se limita a 20 caracteres (`usuario[:20]`) y el texto a 200 caracteres (`texto[:200]`) en el servidor, independientemente de lo que envíe el cliente.
- **Historial circular:** el array de mensajes nunca supera 50 elementos, por lo que un atacante que enviara mensajes en bucle no agotaría la RAM (solo desplazaría los mensajes más antiguos).
- **Buffer de recepción acotado:** `cli.recv(2048)` limita a 2 KB la cantidad de datos que puede enviar un cliente en una sola petición, previniendo ataques de desbordamiento de buffer.
- **Timeout por conexión:** `cli.settimeout(3.0)` evita que un cliente malicioso mantenga una conexión abierta indefinidamente, bloqueando al servidor.
- **Validación de campos:** el router verifica que tanto `usuario` como `texto` sean no vacíos antes de aceptar un mensaje.

10. ¿Cuál es la diferencia entre GET y POST en HTTP?

En el protocolo HTTP, los métodos (o *verbos*) definen la **semántica** de la operación solicitada:

- **GET:** solicita la *lectura* de un recurso. Los parámetros se incluyen en la URL como *query string* (`?clave=valor`). Las peticiones GET son **idempotentes** (repetirlas no cambia el estado del servidor) y están diseñadas para no tener efectos secundarios. En este sistema, GET se usa para obtener la página web (`/`) y para consultar mensajes nuevos (`/mensajes?desde=5`).
- **POST:** envía datos al servidor para *crear o modificar* un recurso. Los datos se incluyen en el **cuerpo** de la petición (no en la URL), lo que permite enviar estructuras más grandes y no expone los datos en la barra de direcciones. Las peticiones POST **no son idempotentes**: repetirlas puede crear duplicados. En este sistema, POST se usa para enviar un nuevo mensaje (`/enviar`).

Esta distinción semántica sigue los principios del **diseño REST** (*Representational State Transfer*): GET para consultas, POST para creación. Aunque en un microcontrolador sería técnicamente posible usar siempre GET, seguir la semántica correcta hace al sistema más predecible y compatible con herramientas estándar de diagnóstico como curl o Postman.

4. Conclusiones

La práctica permitió construir un sistema de comunicación en tiempo real completamente funcional y autónomo partiendo de componentes mínimos: un microcontrolador de \$4, dos archivos de código y ninguna infraestructura externa. El resultado, PicoChat, es un sistema capaz de conectar múltiples dispositivos móviles y permitirles intercambiar mensajes de forma fluida, lo que demuestra que incluso hardware embebido de muy bajo costo puede implementar servicios de red útiles y completos.

Uno de los aprendizajes más significativos fue la implementación manual del servidor HTTP sobre sockets crudos. Al no disponer de un framework de alto nivel como Flask o Express, fue necesario comprender a fondo el formato de las peticiones y respuestas HTTP: la línea de petición, las cabeceras, la separación `\r\n\r\n` entre cabeceras y cuerpo, y los códigos de estado. Este conocimiento de bajo nivel es invaluable porque es transferible a cualquier lenguaje o plataforma.

La gestión de memoria fue otro aprendizaje técnico destacable. El hábito de llamar a `gc.collect()` explícitamente después de cada petición, pre-cargar recursos estáticos en RAM al arrancar, y usar estructuras de datos de tamaño acotado (historial circular) son patrones de diseño propios del desarrollo embebido que no tienen equivalente en el desarrollo de software de escritorio o web convencional, donde la memoria disponible se mide en gigabytes.

El bug del formulario HTML (función `async` devolviendo Promise en lugar de `false`) fue un recordatorio de que la interacción entre tecnologías web (JavaScript moderno con `async/await` y el modelo de eventos del DOM) puede producir comportamientos sutiles e inesperados que requieren conocimiento profundo de los fundamentos para diagnosticarlos correctamente.

Posibles mejoras:

- **Asincronía con uasyncio:** reemplazar el servidor bloqueante por una implementación basada en el módulo `uasyncio` de MicroPython, permitiendo atender múltiples conexiones de forma concurrente sin bloquear el bucle principal.
- **Persistencia en flash:** guardar el historial de mensajes en la flash de la Pico W (por ejemplo, en un archivo JSON) para que sobreviva reinicios del sistema.
- **WebSockets:** implementar soporte de WebSockets (con la librería `uwebsockets` de MicroPython) para eliminar el polling y lograr entrega de mensajes instantánea (*push*).
- **Sala de espera o autenticación:** agregar un mecanismo de autenticación basado en contraseña para que no cualquier dispositivo pueda ver el historial.
- **Notificaciones de escritura:** implementar un indicador de “alguien está escribiendo” mediante un endpoint adicional que reporte actividad reciente del teclado.
- **Modo híbrido AP+STA:** configurar la Pico W en modo simultáneo AP y Station para que también tenga acceso a Internet mientras sirve a los clientes locales, permitiendo reenviar mensajes a un servidor MQTT externo para historial permanente.
- **Interfaz PWA:** agregar un manifiesto `manifest.json` y un Service Worker para que la página web pueda instalarse como aplicación nativa en Android (Progressive Web App), mejorando la experiencia de usuario.
- **Compresión de respuestas:** para reducir el ancho de banda y el tiempo de carga inicial del HTML en redes congestionadas, implementar compresión gzip mediante la cabecera `Content-Encoding: gzip`.

Evidencia Fotográfica

En esta sección se presentan capturas de pantalla del sistema PicoChat en funcionamiento desde dos dispositivos móviles distintos, mostrando el intercambio de mensajes en tiempo real, la diferenciación visual entre mensajes propios y ajenos, el indicador de estado de conexión y la interfaz de usuario completa.

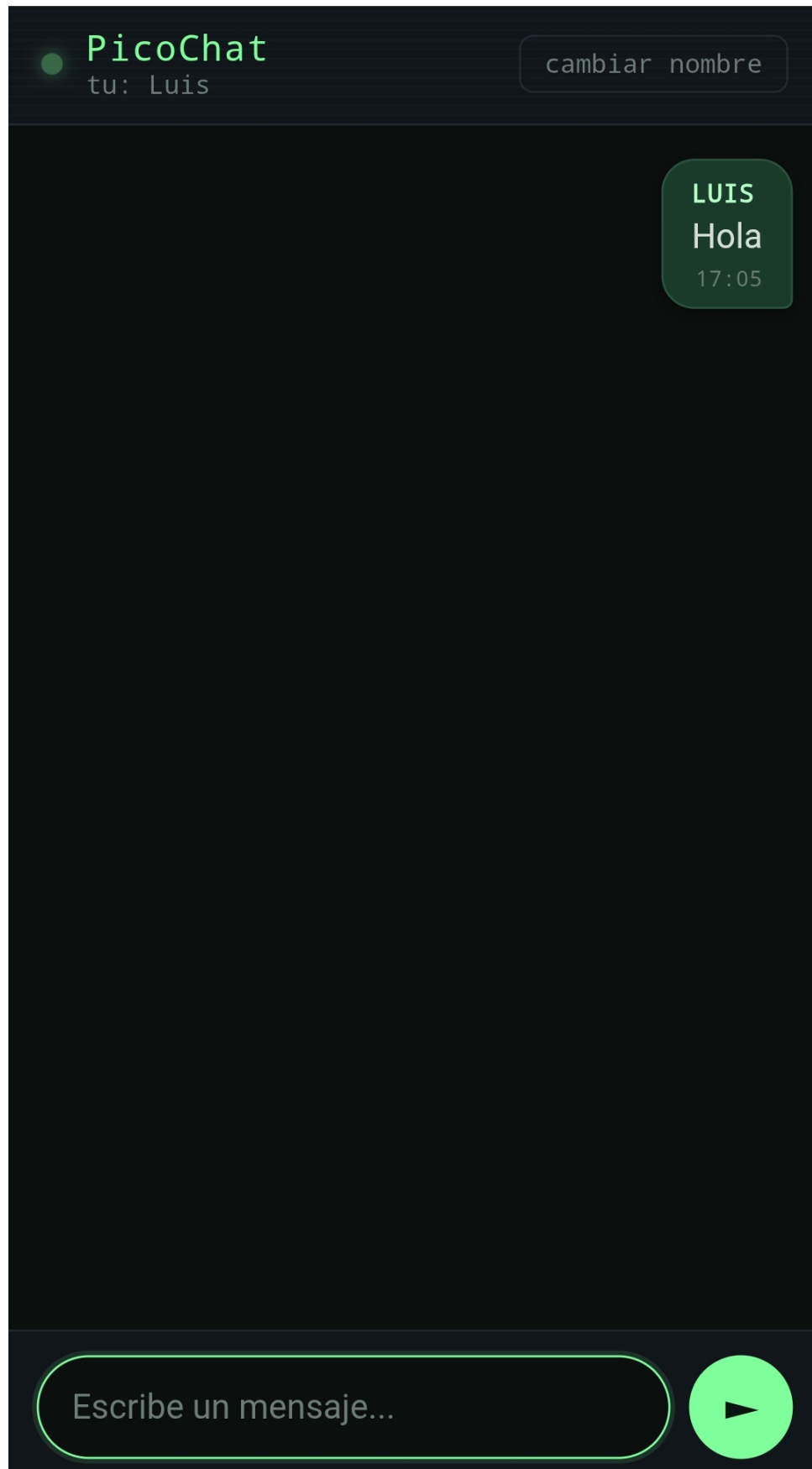


Figura 1: Captura de la interfaz PicoChat en el primer dispositivo (usuario “Saúl”): se observa el historial de mensajes con diferenciación visual entre mensajes propios (derecha, fondo verde) y mensajes del otro usuario (izquierda, fondo oscuro), el indicador de conexión activa (punto verde animado) y el campo de entrada de texto.

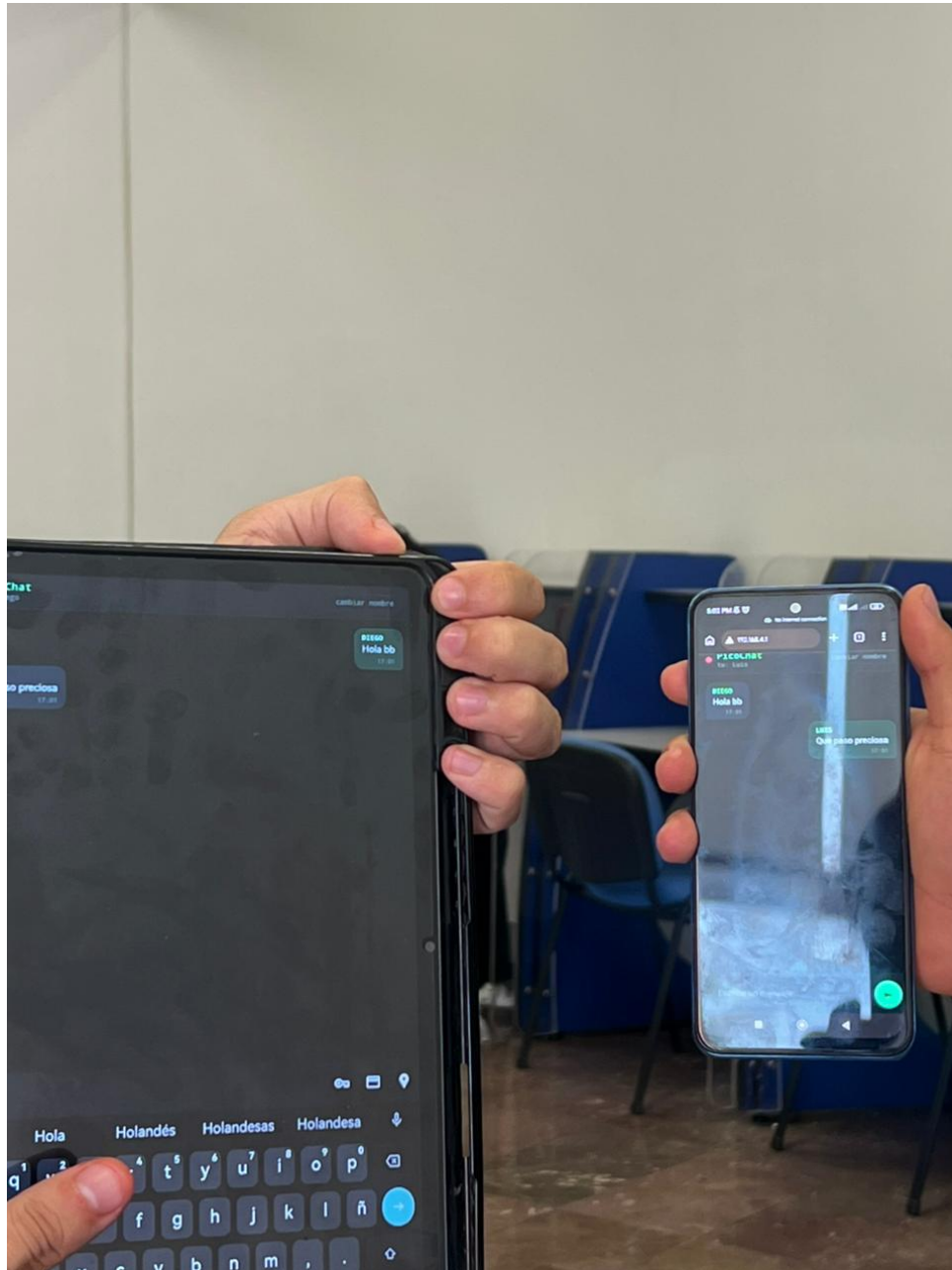


Figura 2: Fotografía de dos dispositivos usando el servicio de Mensajería

Anexos

Anexo A – Archivo completo main.py

```
1  """
2  =====
3  PicoChat - Sistema de mensajería local
```

```

4   Practica 17 - Mecatronica, UASLP-FEPZM
5   Plataforma: Raspberry Pi Pico W con MicroPython
6   =====
7   """
8   import network
9   import socket
10  import ujson as json
11  import time
12  import gc
13  from machine import Pin
14
15  # =====
16  # CONFIGURACION
17  # =====
18  WIFI_SSID      = "PicoChat"
19  WIFI_PASS      = "mecatronica2026"
20  PUERTO         = 80
21  MAX_MENSAJES  = 50
22
23  led = Pin("LED", Pin.OUT)
24
25  # =====
26  # ALMACENAMIENTO DE MENSAJES (en RAM)
27  # =====
28  mensajes      = []
29  contador_id   = 0
30
31  def agregar_mensaje(usuario, texto):
32      global contador_id
33      contador_id += 1
34      msg = {"id": contador_id, "usuario": usuario[:20], "texto"
35            : texto[:200]}
36      mensajes.append(msg)
37      if len(mensajes) > MAX_MENSAJES:
38          mensajes.pop(0)
39      print("[MSG #{}] {}: {}".format(msg["id"], msg["usuario"],
40            msg["texto"]))
41      return msg
42
43  # =====
44  # CONFIGURACION DEL ACCESS POINT
45  # =====
46  def iniciar_ap():
47      ap = network.WLAN(network.AP_IF)
48      ap.config(essid=WIFI_SSID, password=WIFI_PASS)
49      ap.active(True)
50      while not ap.active():
51          time.sleep(0.1)

```

```

50     ip = ap.ifconfig()[0]
51     print("AP activo | IP: {} | URL: http://{0}".format(ip, ip)
52           )
53     return ap
54 # =====
55 # PARSEAR HTTP MINIMO
56 # =====
57 def parsear_peticion(datos):
58     try:
59         texto = datos.decode("utf-8")
60         lineas = texto.split("\r\n")
61         partes = lineas[0].split(" ")
62         if len(partes) < 2: return None, None, None, None
63         metodo = partes[0]
64         url = partes[1]
65         cuerpo = texto.split("\r\n\r\n", 1)[1] if "\r\n\r\n"
66             in texto else ""
67         ruta, query = url.split("?", 1) if "?" in url else (
68             url, ""
69         )
70         return metodo, ruta, query, cuerpo
71     except Exception as e:
72         print("Error parseando peticion:", e)
73         return None, None, None, None
74
75 def parsear_query(query):
76     params = {}
77     if query:
78         for par in query.split("&"):
79             if "=" in par:
80                 k, v = par.split("=", 1)
81                 params[k] = v.replace("+", " ").replace("%20",
82                     " ")
83     return params
84
85 # =====
86 # CONSTRUCCION DE RESPUESTAS HTTP
87 # =====
88 def respuesta_html(cuerpo):
89     return ("HTTP/1.1 200 OK\r\nContent-Type: text/html;
90             charset=utf-8\r\n
91             \"Cache-Control: no-cache\r\nConnection: close\r\n\r\n\" + cuerpo)
92
93 def respuesta_json(data):
94     return ("HTTP/1.1 200 OK\r\nContent-Type: application/json
95             \r\n")

```



```

90         "Access-Control-Allow-Origin: *\r\nConnection:
          close\r\n\r\n"
91         + json.dumps(data))
92
93 def respuesta_404():
94     return ("HTTP/1.1 404 Not Found\r\nContent-Type: text/
          plain\r\n"
95            "Connection: close\r\n\r\n404 - Recurso no
          encontrado")
96
97 # =====
98 # CARGA DE LA PAGINA HTML (una sola vez al arrancar)
99 # =====
100 def cargar_html():
101     with open("index.html", "r") as f:
102         return f.read()
103
104 PAGINA = cargar_html()
105 print("index.html cargado:", len(PAGINA), "bytes")
106
107 # =====
108 # ROUTER
109 # =====
110 def manejar_peticion(metodo, ruta, query, cuerpo):
111     params = parsear_query(query)
112
113     if ruta == "/" and metodo == "GET":
114         return respuesta_html(PAGINA)
115
116     if ruta == "/mensajes" and metodo == "GET":
117         try: desde = int(params.get("desde", "0"))
118         except ValueError: desde = 0
119         nuevos = [m for m in mensajes if m["id"] > desde]
120         return respuesta_json({"mensajes": nuevos, "ultimo":
          contador_id})
121
122     if ruta == "/enviar" and metodo == "POST":
123         try:
124             data = json.loads(cuerpo)
125             usuario = (data.get("usuario") or "Anonimo").strip()
126             texto = (data.get("texto") or "").strip()
127             if usuario and texto:
128                 msg = agregar_mensaje(usuario, texto)
129                 led.toggle()
130                 return respuesta_json({"ok": True, "id": msg["
          id"]})

```

```

131         return respuesta_json({"ok": False, "error": "
132             datos invalidos"})
133     except Exception as e:
134         return respuesta_json({"ok": False, "error": str(e
135             )})
136
137     return respuesta_404()
138
139 # =====
140 # SERVIDOR HTTP
141 # =====
142 def iniciar_servidor():
143     addr = socket.getaddrinfo("0.0.0.0", PUERTO)[0][-1]
144     s = socket.socket()
145     s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
146     s.bind(addr)
147     s.listen(5)
148     print("Servidor HTTP escuchando en puerto", PUERTO)
149
150     while True:
151         cli = None
152         try:
153             cli, _ = s.accept()
154             cli.settimeout(3.0)
155             datos = cli.recv(2048)
156             if datos:
157                 metodo, ruta, query, cuerpo = parsear_peticion
158                     (datos)
159                 if ruta is not None:
160                     respuesta = manejar_peticion(metodo, ruta,
161                         query, cuerpo)
162                     cli.sendall(respuesta.encode("utf-8"))
163             except Exception as e:
164                 print("Error de conexi3n:", e)
165             finally:
166                 if cli is not None:
167                     try: cli.close()
168                     except: pass
169                 gc.collect()
170
171 # =====
172 # ARRANQUE
173 # =====
174 if __name__ == "__main__":
175     iniciar_ap()
176     led.on()
177     try:
178         iniciar_servidor()

```

```

175     except KeyboardInterrupt:
176         print("Servidor detenido manualmente")
177         led.off()

```

Anexo B – Archivo completo index.html

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width,
6      initial-scale=1.0, maximum-scale=1.0">
7  <title>PicoChat</title>
8  <style>
9  :root {
10     --bg: #0b0f0d; --panel: #11161a; --line: #1f2a30;
11     --text: #d6e0db; --muted: #6b7d76; --accent: #7fff9c;
12     --propio: #1a3a2a; --ajeno: #1c2630;
13  }
14  /* ... (estilos completos en el archivo fuente) ... */
15  </style>
16  </head>
17  <body>
18  <header>
19     <div class="dot" id="estado"></div>
20     <div class="titulo">
21         <h1>PicoChat</h1>
22         <small id="info">conectando...</small>
23     </div>
24     <button class="btn-nombre" id="btnNombre">cambiar nombre</
25         button>
26 </header>
27 <div id="chat">
28     <div class="aviso">
29         Servidor: Raspberry Pi Pico W<br>Esperando mensajes...
30     </div>
31 </div>
32 <form id="form">
33     <input id="texto" type="text" placeholder="Escribe un
34         mensaje..."
35         autocomplete="off" maxlength="200" required>
36     <button id="enviar" type="submit">#9658;</button>
37 </form>
38 <script>
39 let usuario = localStorage.getItem("picochat_usuario");
40 if (!usuario) {

```

```
39     usuario = (prompt("Como te llamas?") || "Anonimo")
40               .trim().slice(0, 20) || "Anonimo";
41     localStorage.setItem("picochat_usuario", usuario);
42 }
43 let ultimoId = 0;
44
45 function actualizar() {
46     return fetch("/mensajes?desde=" + ultimoId)
47       .then(r => r.json())
48       .then(d => {
49         d.mensajes.forEach(pintar);
50         if (d.ultimo > ultimoId) ultimoId = d.ultimo;
51       });
52 }
53
54 function enviar(texto) {
55     return fetch("/enviar", {
56       method: "POST",
57       headers: {"Content-Type": "application/json"},
58       body: JSON.stringify({usuario, texto})
59     }).then(() => actualizar());
60 }
61
62 document.getElementById("form").addEventListener("submit", ev
63   => {
64     ev.preventDefault();
65     const input = document.getElementById("texto");
66     const texto = input.value.trim();
67     if (!texto) return;
68     input.value = "";
69     enviar(texto);
70 });
71
72 actualizar();
73 setInterval(actualizar, 1500);
74 </script>
75 </body>
76 </html>
```

Referencias

- [1] MicroPython, *Quick reference for the Raspberry Pi Pico W*. <https://docs.micropython.org/en/latest/rp2/quickref.html>, 2024.
- [2] MicroPython, *network – network configuration*. <https://docs.micropython.org/en/latest/library/network.html>, 2024.

- [3] MicroPython, *usocket – socket module*. <https://docs.micropython.org/en/latest/library/usocket.html>, 2024.
- [4] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.
- [5] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.
- [6] Infineon Technologies, *CYW43439 Single-Chip 2.4 GHz IEEE 802.11 b/g/n WLAN + Bluetooth 5.0 Combo Chip*. <https://www.infineon.com/cms/en/product/wireless-connectivity/airoc-wi-fi-plus-bluetooth-combos/cyw43439/>, 2023.
- [7] R. Fielding et al., *RFC 7230 – Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing*. IETF, 2014. <https://www.rfc-editor.org/rfc/rfc7230>
- [8] T. Bray (ed.), *RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format*. IETF, 2017. <https://www.rfc-editor.org/rfc/rfc8259>
- [9] MicroPython, *gc – control the garbage collector*. <https://docs.micropython.org/en/latest/library/gc.html>, 2024.